



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

"МИРЭА - Российский технологический университет"

РТУ МИРЭА

Институт информационных технологий (ИТ)

Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №6

по дисциплине

«Разработка баз данных»

Выполнил студент группы ИНБО-20-23

Школьный И.В.

Проверил доцент

Исаева М.В.

Москва 2025

ПРАКТИЧЕСКАЯ РАБОТА №6. ТРИГГЕРЫ И КУРСОРЫ В POSTGRESQL

Цель работы:

Целью данной практической работы является формирование у студентов углубленных практических навыков по управлению данными и реализации сложной бизнес-логики в СУБД PostgreSQL с использованием триггеров и курсоров.

Постановка задачи:

Для выполнения практической работы необходимо последовательно выполнить два задания, демонстрирующих применение триггеров и курсоров. Ваша задача — адаптировать каждую из поставленных задач к логической структуре и предметной области вашей базы данных.

Задание №1: триггеры

Проанализировать предметную область своей базы данных и выявить не менее трёх бизнес-правил, реализация которых в виде ограничений целостности возможна только с помощью триггеров. ВАЖНО: триггеры должны быть разными.

Задание №2: курсоры Разработать два скрипта на PL/pgSQL, демонстрирующих оба способа обработки данных:

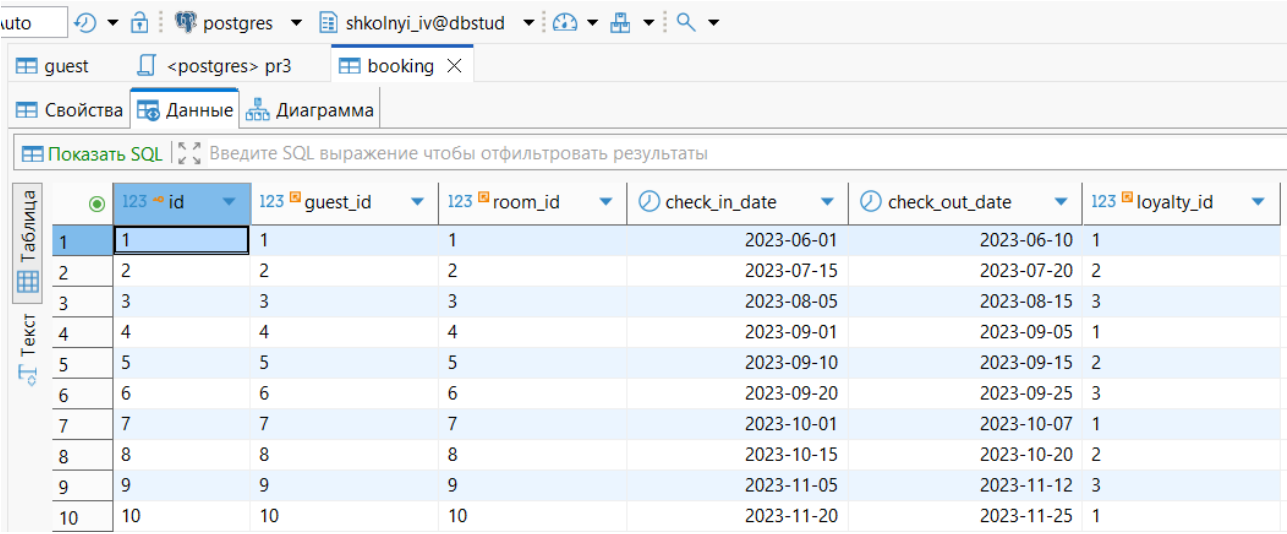
- С использованием явного курсора (DECLARE/OPEN/FETCH/CLOSE).
- С использованием неявного курсора (цикл FOR...IN).

ХОД ВЫПОЛНЕНИЯ РАБОТЫ

Для создания понятных и последовательных примеров в рамках данной работы будет использоваться упрощенная схема базы данных «Гостиницы». Эта схема послужит основой для демонстрации работы триггеров и курсоров. Студентам следует адаптировать представленные примеры для своих собственных баз данных.

Ниже приводятся таблицы, используемые для построения запросов.

Таблица 1. Таблица Бронирование (Booking)



	123 id	123 guest_id	123 room_id	check_in_date	check_out_date	123 loyalty_id
1	1	1	1	2023-06-01	2023-06-10	1
2	2	2	2	2023-07-15	2023-07-20	2
3	3	3	3	2023-08-05	2023-08-15	3
4	4	4	4	2023-09-01	2023-09-05	1
5	5	5	5	2023-09-10	2023-09-15	2
6	6	6	6	2023-09-20	2023-09-25	3
7	7	7	7	2023-10-01	2023-10-07	1
8	8	8	8	2023-10-15	2023-10-20	2
9	9	9	9	2023-11-05	2023-11-12	3
10	10	10	10	2023-11-20	2023-11-25	1

Таблица 2. Таблица Документы о бронировании (Booking_document)

auto postgres shkolnyi_iv@dbstud

booking_document

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

	123 id	123 booking_id	issue_date	A-Z status
1	1	1	2023-05-15	Confirmed
2	2	2	2023-06-20	Paid
3	3	3	2023-07-10	Completed
4	4	4	2023-08-15	Confirmed
5	5	5	2023-08-25	Paid
6	6	6	2023-09-10	Completed
7	7	7	2023-09-20	Confirmed
8	8	8	2023-10-05	Paid
9	9	9	2023-10-25	Completed
10	10	10	2023-11-10	Confirmed

Таблица 3. Таблица Брошюра (Brochure)

auto postgres shkolnyi_iv@dbstud

brochure

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

	123 id	123 hotel_id	A-Z language	publication_date
1	1	1	Russian	2022-01-10
2	2	2	English	2022-03-15
3	3	3	German	2022-05-20
4	4	1	English	2022-02-28
5	5	2	German	2022-04-10
6	6	3	Russian	2022-06-15
7	7	1	German	2022-07-20
8	8	2	English	2022-08-25
9	9	3	English	2022-09-30
10	10	1	Russian	2022-10-05
11	11	2	Russian	2022-11-12
12	12	3	German	2022-12-18

Таблица 4. Таблица Регистрация (Check_in)

Auto postgres shkolnyi_iv@dbstud

check_in ×

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

Таблица	123 id	123 guest_id	123 total_bill	123 room_id	123 doc_id
1	1	1	45 000	1	1
2	2	2	42 500	2	2
3	3	3	120 000	3	3
4	4	4	18 000	4	4
5	5	5	39 000	5	5
6	6	6	57 500	6	6
7	7	7	31 200	7	7
8	8	8	41 000	8	8
9	9	9	84 000	9	9
10	10	10	27 500	10	10

Таблица 5. Таблица Работник (Employee)

Auto postgres shkolnyi_iv@dbstud

employee ×

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

Таблица	123 id	A-Z full_name	123 position_id	hire_date	123 hotel_id
1	1	Смирнова Ольга Дмитриевна	1	2018-06-10	1
2	2	Кузнецов Андрей Петрович	2	2020-02-15	2
3	3	Васильева Елена Ивановна	3	2021-05-20	3
4	4	Петров Максим Игоревич	4	2019-03-12	1
5	5	Семенова Ирина Васильевна	5	2020-08-25	2
6	6	Ковалев Александр Петрович	6	2018-11-30	3
7	7	Новикова Юлия Сергеевна	7	2021-02-14	1
8	8	Данилов Алексей Викторович	8	2017-07-22	2
9	9	Тихонова Марина Олеговна	9	2016-05-18	3
10	10	Фролов Денис Николаевич	10	2022-01-10	1

Таблица 6. Таблица Гость (Guest)

Auto postgres shkolnyi_iv@dbstud

guest X

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

Таблица	123 id	A-Z full_name	A-Z phone_number	A-Z email
1	1	Козлова Мария Игоревна	7911123456	kozlova@mail.ru
2	2	Николаев Дмитрий Олегович	7902876543	nikolaev@gmail.com
3	3	Федорова Екатерина Павловна	7922555777	fedorova@yandex.ru
4	4	Громов Сергей Александрович	7913987654	gromov@mail.ru
5	5	Волкова Анастасия Викторовна	7904765432	volkova@gmail.com
6	6	Белов Артем Сергеевич	7923876541	belov@yandex.ru
7	7	Соколова Ольга Николаевна	7915234567	sokolova@mail.ru
8	8	Иванов Иван Иванович	7910123456	ivanov@mail.ru
9	9	Петрова Анна Сергеевна	7909876543	petrova@gmail.com
10	10	Сидоров Алексей Викторович	7921555777	sidorov@yandex.ru

Таблица 7. Таблица Документы гостя (Guest_document)

uto postgres shkolnyi_iv@dbstud

guest_document X

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

Таблица	123 id	123 guest_id	A-Z document_data	A-Z document_type	issue_date	expiry_date
1	1	1	1234567890	Passport	2015-05-20	2025-05-20
2	2	2	0987654321	Birth certificate	2020-01-15	2030-01-15
3	3	3	1122334455	Driver license	2018-11-30	2028-11-30
4	4	4	4455667788	Passport	2017-03-10	2027-03-10
5	5	5	5566778899	Driver license	2019-08-22	2029-08-22
6	6	6	6677889900	Passport	2016-11-05	2026-11-05
7	7	7	7788990011	Birth certificate	2021-02-14	2031-02-14
8	8	8	8899001122	Driver license	2018-07-30	2028-07-30
9	9	9	9900112233	Passport	2019-12-20	2029-12-20
10	10	10	1011223344	Birth certificate	2020-04-18	2030-04-18

Таблица 8. Таблица Гостиница (Hotel)

uto postgres shkolnyi_iv@dbstud

guest_document hotel X

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

Таблица	123 id	A-Z name	A-Z address	123 star_rating	A-Z description
1	1	City Lights	ул. Невский проспект, 45	4	Современный отель в историческом центре
2	2	River Side	наб. реки Мойки, 12	5	Элитный отель с видом на реку
3	3	Park Inn	ул. Садовая, 78	3	Комфортабельный отель рядом с парком

Таблица 9. Таблица Программа лояльности (Loyalty_program)

auto postgres shkolnyi_iv@dbstud

loyalty_program

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

123 id	A-Z name	123 discount	A-Z description
1	Gold Member	15	Exclusive discounts and early check-in for gold members
2	Silver Member	10	Special offers and room upgrades for silver members
3	Bronze Member	5	Basic discounts for frequent guests

Таблица 10. Таблица Должность (Position)

auto postgres shkolnyi_iv@dbstud

loyalty_program position

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

123 id	A-Z title	123 base_rate
1	Менеджер	50 000
2	Портье	35 000
3	Горничная	30 000
4	Администратор	45 000
5	Шеф-повар	80 000
6	Официант	32 000
7	Охранник	38 000
8	Уборщик	28 000
9	Бухгалтер	55 000
10	Маркетолог	48 000
11	IT-специалист	60 000
12	Директор	100 000
13	Аниматор	35 000

Таблица 11. Таблица Номер (Room)

auto

postgres

shkolnyi_iv@dbstud

loyalty_program

position

room X

Свойства

Данные

Диаграмма

Показать SQL

Введите SQL выражение чтобы отфильтровать результаты

	123 id	A-Z room_number	A-Z type	123 price_per_night	123 hotel_id
1	1	101	Standard	5 000	1
2	2	202	Deluxe	8 500	2
3	3	301	Suite	12 000	3
4	4	102	Standard	4 500	1
5	5	103	Deluxe	7 500	1
6	6	201	Standard	4 800	2
7	7	202	Deluxe	7 800	2
8	8	203	Suite	11 500	2
9	9	301	Standard	5 200	3
10	10	302	Deluxe	8 200	3
11	11	303	Suite	12 500	3
12	12	401	Standard	5 500	1

Задание 1. Триггеры

Сперва необходимо сформировать бизнес-правила, по которым будут строиться триггеры.

Проверка доступности номеров. При бронировании номера необходимо гарантировать, что он не занят другими гостями на выбранные даты. Напишем триггер, проверяющий доступность номера перед его бронированием.

Листинг 1. Функция триггера — проверка доступности номера

```
-- Создание функции триггера-1
CREATE OR REPLACE FUNCTION validate_room_availability()
RETURNS TRIGGER AS $$
DECLARE
    overlapping_bookings INT;
BEGIN
    -- Проверяем пересекающиеся бронирования
    SELECT COUNT(*) INTO overlapping_bookings
    FROM booking b
    WHERE b.room_id = NEW.room_id
        AND b.check_in_date <= NEW.check_out_date
        AND b.check_out_date >= NEW.check_in_date
        AND b.id != COALESCE(NEW.id, -1);

    -- Если есть пересечения - отменяем операцию
    IF overlapping_bookings > 0 THEN
        RAISE EXCEPTION 'Номер % занят на указанные даты', NEW.room_id;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Запустим скрипт.

Commit ▾ Rollback ▾ T ▾ Auto ↺ 🔒 postgres shkolyi_iv@dbstud ▾		
*<postgres> pr6 X		
<pre> -- Если есть пересечения - отменяем операцию IF overlapping_bookings > 0 THEN RAISE EXCEPTION 'Номер % занят на указанные даты', NEW.room_id; END IF; RETURN NEW; END;</pre>		
Статистика 1 X		
Name	Value	
Updated Rows	0	
Execute time	0.011s	
Start time	Mon Nov 10 21:14:06 MSK 2025	
Finish time	Mon Nov 10 21:14:31 MSK 2025	
Query	<pre> -- Создание функции триггера-1 CREATE OR REPLACE FUNCTION validate_room_availability() RETURNS TRIGGER AS \$\$ DECLARE overlapping_bookings INT; BEGIN -- Проверяем пересекающиеся бронирования SELECT COUNT(*) INTO overlapping_bookings FROM booking b WHERE b.room_id = NEW.room_id AND b.check_in_date <= NEW.check_out_date AND b.check_out_date >= NEW.check_in_date AND b.id != COALESCE(NEW.id, -1); -- Если есть пересечения - отменяем операцию IF overlapping_bookings > 0 THEN RAISE EXCEPTION 'Номер % занят на указанные даты', NEW.room_id; END IF; RETURN NEW; END; \$\$ LANGUAGE plpgsql</pre>	

Рисунок 1 — Созданная функция триггера

Теперь создадим непосредственно триггер.

Листинг 2. Триггер — проверка доступности номера

```

-- Создание триггера-1
CREATE TRIGGER availability_trigger
BEFORE INSERT OR UPDATE ON booking
FOR EACH ROW
EXECUTE FUNCTION validate_room_availability();
```

Запустим скрипт.

Commit ▾ Rollback ▾ T ▾ Auto ↺ 🔒 postgres shkolyi_iv@dbstud

*<postgres> pr6 ×

▶ + 📄 AI > ⚙

```
$$ LANGUAGE plpgsql;

-- Создание триггера-1
CREATE TRIGGER availability_trigger
  BEFORE INSERT OR UPDATE ON booking
  FOR EACH ROW
  EXECUTE FUNCTION validate_room_availability();
```

Статистика 1 ×

Name	Value
Updated Rows	0
Execute time	0.010s
Start time	Mon Nov 10 21:24:57 MSK 2025
Finish time	Mon Nov 10 21:24:57 MSK 2025
Query	-- Создание триггера-1
	CREATE TRIGGER availability_trigger
	BEFORE INSERT OR UPDATE ON booking
	FOR EACH ROW
	EXECUTE FUNCTION validate_room_availability()

Рисунок 2 — Созданный триггер

Теперь протестируем его работу. Для этого запустим скрипты добавления записи, один из которых завершится успешно, а другой — неудачно.

Листинг 3. Тесты первого триггера

```
-- УСПЕШНО: Номер свободен
INSERT INTO booking (guest_id, room_id, check_in_date, check_out_date, loyalty_id)
VALUES (1, 1, '2024-02-01', '2024-02-05', 1);

-- ОШИБКА: Номер уже занят
INSERT INTO booking (guest_id, room_id, check_in_date, check_out_date, loyalty_id)
VALUES (2, 1, '2024-02-03', '2024-02-07', 2);
```

И действительно, первый запрос выполнен успешно, а на втором произошла ошибка. Это подтверждает корректную работу триггера.

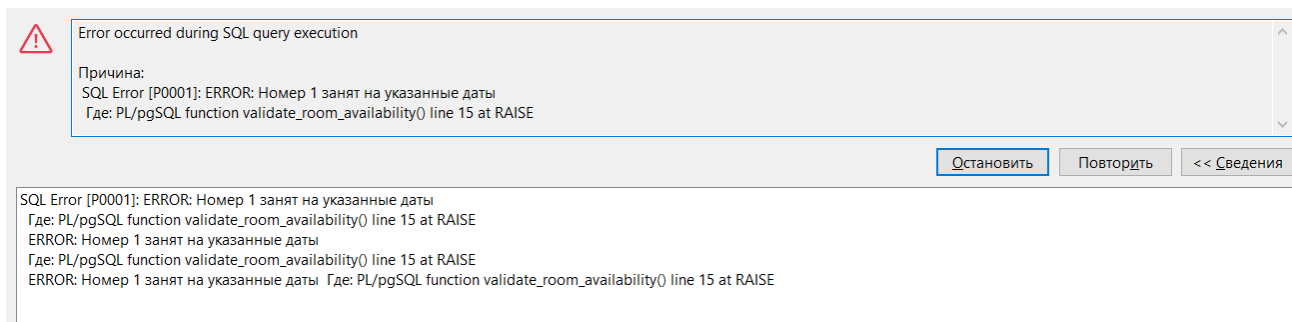


Рисунок 3 — Неудачный запрос

Автоматический расчет стоимости с учетом скидок. При регистрации заезда нужно автоматически применять скидки программ лояльности. Напишем триггер, вычисляющий итоговую стоимость с учетом скидок.

Листинг 4. Функция триггера — расчет итоговой стоимости

```
CREATE OR REPLACE FUNCTION apply_loyalty_discount()  
RETURNS TRIGGER AS $$  
DECLARE  
    nightly_price DOUBLE PRECISION;  
    stay_duration INT;  
    discount_rate DOUBLE PRECISION;  
    total_without_discount DOUBLE PRECISION;  
    booking_loyalty_id INT;  
    booking_check_in DATE;  
    booking_check_out DATE;  
BEGIN  
    -- Получаем цену номера за ночь  
    SELECT price_per_night INTO nightly_price  
    FROM room WHERE id = NEW.room_id;  
  
    -- Получаем ID программы лояльности и даты бронирования через связь таблиц  
    SELECT  
        b.loyalty_id,  
        b.check_in_date,  
        b.check_out_date  
    INTO  
        booking_loyalty_id,  
        booking_check_in,  
        booking_check_out  
    FROM booking_document bd  
    JOIN booking b ON bd.booking_id = b.id  
    WHERE bd.id = NEW.doc_id;  
  
    -- Получаем размер скидки из программы лояльности  
    SELECT discount INTO discount_rate  
    FROM loyalty_program WHERE id = booking_loyalty_id;  
  
    -- Рассчитываем продолжительность проживания  
    stay_duration := booking_check_out - booking_check_in;
```

```
-- Рассчитываем итоговый счет  
total_without_discount := nightly_price * stay_duration;  
NEW.total_bill := total_without_discount * (1 - discount_rate / 100);  
  
RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

Функция успешно создана.

Rollback ▼ T Auto ↺ 🔒 postgres shkolnyi_iv@dbstud 🔄 📄	
* <postgres> pr6 × booking check_in booking_document	
Статистика 1 ×	
Name	Value
Updated Rows	0
Execute time	0.013s
Start time	Mon Nov 10 22:32:36 MSK 2025
Finish time	Mon Nov 10 22:32:36 MSK 2025
Query	CREATE OR REPLACE FUNCTION apply_loyalty_discount() RETURNS TRIGGER AS \$\$ DECLARE nightly_price DOUBLE PRECISION; stay_duration INT; discount_rate DOUBLE PRECISION; total_without_discount DOUBLE PRECISION; booking_loyalty_id INT; booking_check_in DATE; booking_check_out DATE; BEGIN -- Получаем цену номера за ночь SELECT price_per_night INTO nightly_price FROM room WHERE id = NEW.room_id; -- Получаем ID программы лояльности и даты бронирования через связь таблиц SELECT b.loyalty_id, b.check_in_date, b.check_out_date INTO booking_loyalty_id, booking_check_in, booking_check_out FROM booking_document bd JOIN booking b ON bd.booking_id = b.id WHERE bd.id = NEW.doc_id; -- Получаем размер скидки из программы лояльности SELECT discount INTO discount_rate FROM loyalty_program WHERE id = booking_loyalty_id; -- Рассчитываем продолжительность проживания

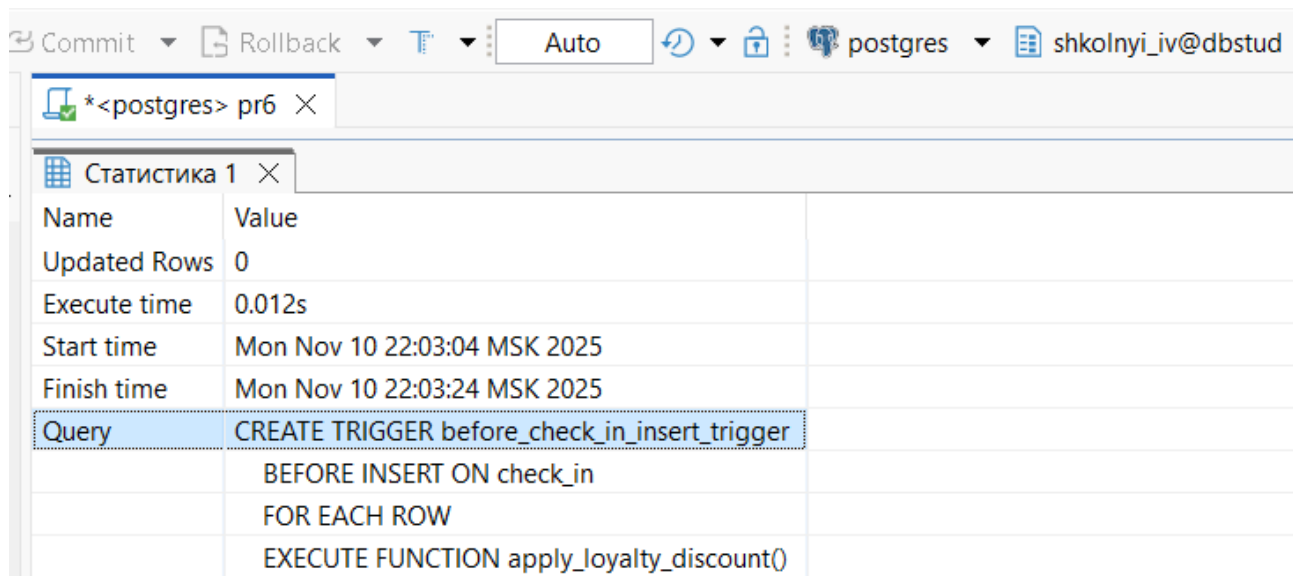
Рисунок 4 — Созданная функция триггера

Теперь напишем скрипт создания триггера.

Листинг 5. Функция триггера — расчет итоговой стоимости

```
CREATE TRIGGER before_check_in_insert_trigger
BEFORE INSERT ON check_in
FOR EACH ROW
EXECUTE FUNCTION apply_loyalty_discount();
```

Триггер успешно создан.



The screenshot shows a PostgreSQL client window with a toolbar at the top containing buttons for Commit, Rollback, and Auto, along with a refresh icon and a lock icon. The window title is '*<postgres> pr6'. Below the toolbar, there is a tab labeled 'Статистика 1'. The main area displays a table with two columns: 'Name' and 'Value'. The table contains the following rows:

Name	Value
Updated Rows	0
Execute time	0.012s
Start time	Mon Nov 10 22:03:04 MSK 2025
Finish time	Mon Nov 10 22:03:24 MSK 2025
Query	CREATE TRIGGER before_check_in_insert_trigger BEFORE INSERT ON check_in FOR EACH ROW EXECUTE FUNCTION apply_loyalty_discount()

Рисунок 5 — Созданный триггер

Протестируем работу триггера. Последовательно добавим записи в таблицы бронирования, документов о бронировании и регистрации.

Листинг 6. Тест триггера-2

```
-- Создаем тестовое бронирование
INSERT INTO booking (id, guest_id, room_id, check_in_date, check_out_date, loyalty_id)
VALUES (12, 1, 1, '2024-02-01', '2024-02-04', 1);
INSERT INTO booking_document (id, booking_id, issue_date, status)
VALUES (12, 12, '2024-01-20', 'Confirmed');
-- Вставляем запись регистрации (total_bill передаем как 0 - будет пересчитан триггером)
INSERT INTO check_in (id, guest_id, total_bill, room_id, doc_id)
VALUES (11, 1, 0, 1, 12);
```

Какой ответ мы ожидаем получить: $(5000 * 3) * (1 - 0.18) = 12300$.

	123 id	123 guest_id	123 total_bill	123 room_id	123 doc_id
1	1	1	45 000	1	1
2	2	2	42 500	2	2
3	3	3	120 000	3	3
4	4	4	18 000	4	4
5	5	5	39 000	5	5
6	6	6	57 500	6	6
7	7	7	31 200	7	7
8	8	8	41 000	8	8
9	9	9	84 000	9	9
10	10	10	27 500	10	10
11	11	1	12 300	1	12

Рисунок 6 — Корректная работа

Аудит изменений цен номеров. В гостинице необходимо отслеживать все изменения цен номеров для анализа ценовой политики, контроля за действиями сотрудников, исторической отчетности.

Аудит — это процесс отслеживания и регистрации всех изменений данных. Сперва необходимо создать таблицу аудита.

Листинг 7. Таблица для аудита

```
CREATE TABLE room_price_audit (  
  id SERIAL PRIMARY KEY,  
  room_id INT NOT NULL,  
  old_price DOUBLE PRECISION NOT NULL,  
  new_price DOUBLE PRECISION NOT NULL,  
  change_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  changed_by TEXT DEFAULT CURRENT_USER  
);
```

Напишем функцию для триггера.

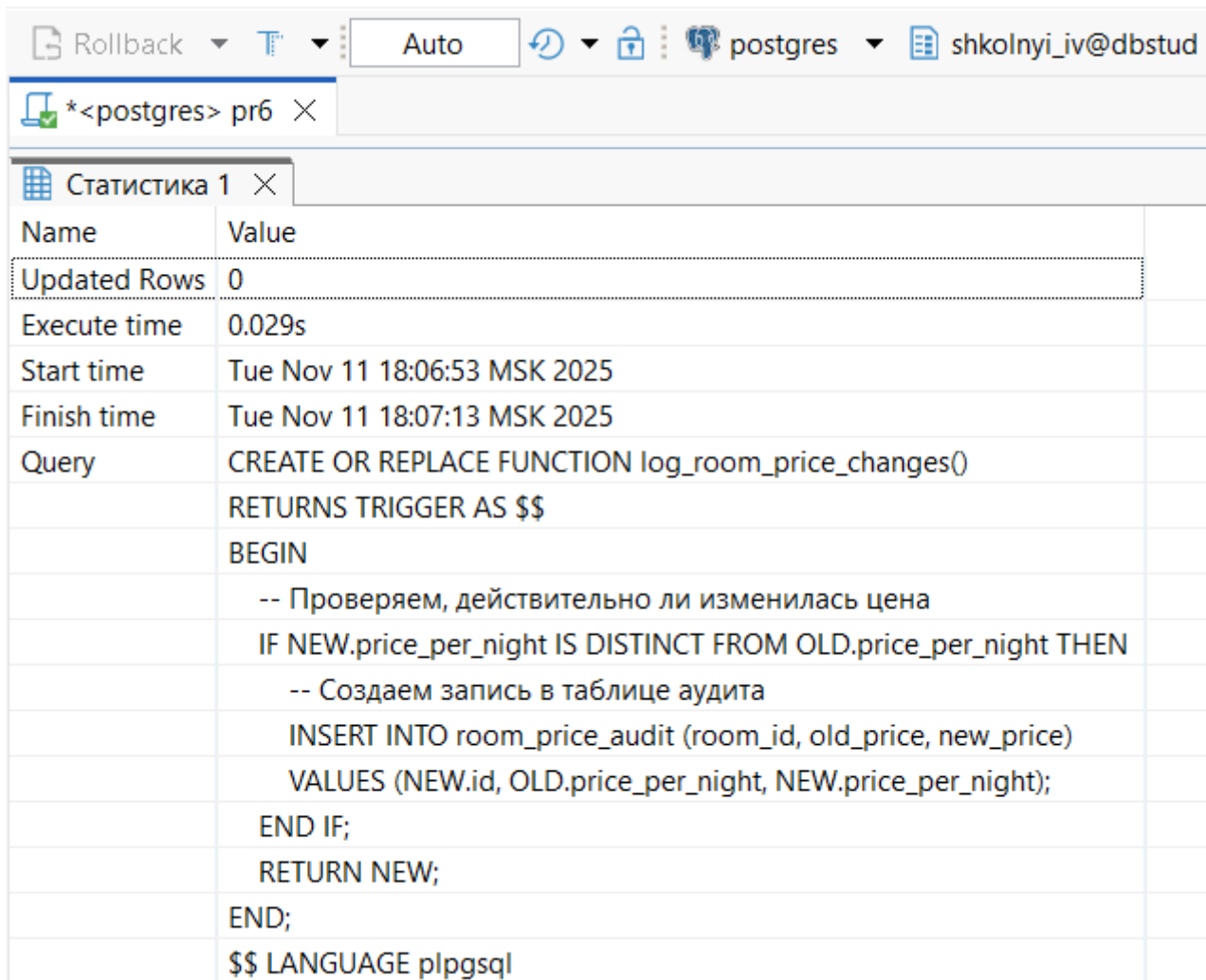
Листинг 8. Функция триггера

```
CREATE OR REPLACE FUNCTION log_room_price_changes()  
RETURNS TRIGGER AS $$  
BEGIN  
  -- Проверяем, действительно ли изменилась цена  
  IF NEW.price_per_night IS DISTINCT FROM OLD.price_per_night THEN  
    -- Создаем запись в таблице аудита  
    INSERT INTO room_price_audit (room_id, old_price, new_price)
```

```
VALUES (NEW.id, OLD.price_per_night, NEW.price_per_night);  
END IF;
```

```
RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

Функция успешно создана.



The screenshot shows a PostgreSQL client window with a toolbar at the top containing icons for Rollback, text formatting, Auto, refresh, lock, and database selection (postgres). Below the toolbar is a tab labeled '*<postgres> pr6'. A 'Статистика 1' (Statistics 1) window is open, displaying a table with execution details.

Name	Value
Updated Rows	0
Execute time	0.029s
Start time	Tue Nov 11 18:06:53 MSK 2025
Finish time	Tue Nov 11 18:07:13 MSK 2025
Query	CREATE OR REPLACE FUNCTION log_room_price_changes() RETURNS TRIGGER AS \$\$ BEGIN -- Проверяем, действительно ли изменилась цена IF NEW.price_per_night IS DISTINCT FROM OLD.price_per_night THEN -- Создаем запись в таблице аудита INSERT INTO room_price_audit (room_id, old_price, new_price) VALUES (NEW.id, OLD.price_per_night, NEW.price_per_night); END IF; RETURN NEW; END; \$\$ LANGUAGE plpgsql

Рисунок 7 — Успешное создание функции триггера

Напишем скрипт создания триггера.

Листинг 10. Триггер

```
CREATE TRIGGER room_price_update_trigger  
AFTER UPDATE ON room  
FOR EACH ROW  
EXECUTE FUNCTION log_room_price_changes();
```

Триггер успешно создан.

Name	Value
Updated Rows	0
Execute time	0.012s
Start time	Tue Nov 11 18:10:13 MSK 2025
Finish time	Tue Nov 11 18:10:13 MSK 2025
Query	CREATE TRIGGER room_price_update_trigger AFTER UPDATE ON room FOR EACH ROW EXECUTE FUNCTION log_room_price_changes()

Рисунок 8 — Успешно созданный триггер

Перейдем к тестированию триггера. Для этого изменим цену первой комнаты с 5000 на 6000. Затем проверим запись в аудите.

Листинг 11. Тестирование триггера 3

```
UPDATE room SET price_per_night = 6000 WHERE id = 1;
```

Триггер работает корректно.

id	room_id	old_price	new_price	change_date	changed_by
1	1	5 000	6 000	2025-11-11 18:23:16.531	shkolnyi_iv

Рисунок 9 — Корректная работа триггера

Задание 2. Курсоры

Курсор — это объект базы данных, который позволяет обрабатывать результаты SQL-запроса построчно, как в цикле программирования.

Допустим, необходимо найти всех гостей с документами, которые истекают в ближайшие 30 дней, и сгенерировать для них персонализированные уведомления.

Листинг 12. Явный курсор

```
-- Явный курсор: Обработка гостей с истекающими документами
```

DO \$\$

DECLARE

-- 1. ОБЪЯВЛЕНИЕ КУРСОРА - связываем с запросом

guest_cursor **CURSOR FOR**

SELECT

g.id,

g.full_name,

g.email,

gd.document_type,

gd.expiry_date,

(gd.expiry_date - CURRENT_DATE) as days_until_expiry

FROM *guest g*

JOIN *guest_document gd* **ON** *g.id = gd.guest_id*

ORDER BY *gd.expiry_date;*

-- Переменные для хранения данных из курсора

guest_id **BIGINT**;

guest_name **VARCHAR**(30);

guest_email **VARCHAR**(20);

doc_type **VARCHAR**(30);

expiry_date **DATE**;

days_left **INT**;

BEGIN

-- 2. ОТКРЫТИЕ КУРСОРА - выполнение запроса

OPEN *guest_cursor*;

-- 3. ЦИКЛ ОБРАБОТКИ ДАННЫХ

LOOP

-- 3.1. ИЗВЛЕЧЕНИЕ ДАННЫХ - получаем следующую строку

FETCH *guest_cursor* **INTO** *guest_id, guest_name, guest_email, doc_type, expiry_date, days_left*;

-- 3.2. ПРОВЕРКА КОНЦА ДАННЫХ

EXIT WHEN NOT FOUND;

-- 3.3. ОБРАБОТКА ДАННЫХ - бизнес-логика

IF *days_left* <= 0 **THEN**

RAISE NOTICE 'СРОЧНО! Гость % (ID:%), документ необходимо обновить!!',

guest_name, guest_id;

ELSIF *days_left* <= 7 **THEN**

RAISE NOTICE 'СРОЧНО! Гость % (ID:%), документ % истекает через % дней. Email: %',

guest_name, guest_id, doc_type, days_left, guest_email;

ELSIF *days_left* <= 14 **THEN**

RAISE NOTICE 'Внимание! Гость % (ID:%), документ % истекает через % дней. Email: %',

guest_name, guest_id, doc_type, days_left, guest_email;

ELSE

RAISE NOTICE 'Напоминание: Гость % (ID:%), документ % истекает через % дней. Email: %',

```

        guest_name, guest_id, doc_type, days_left, guest_email;
    END IF;

END LOOP;

-- 4. ЗАКРЫТИЕ КУРСОРА - освобождение ресурсов
CLOSE guest_cursor;

END;
$$ LANGUAGE plpgsql;

```

Курсор успешно работает.

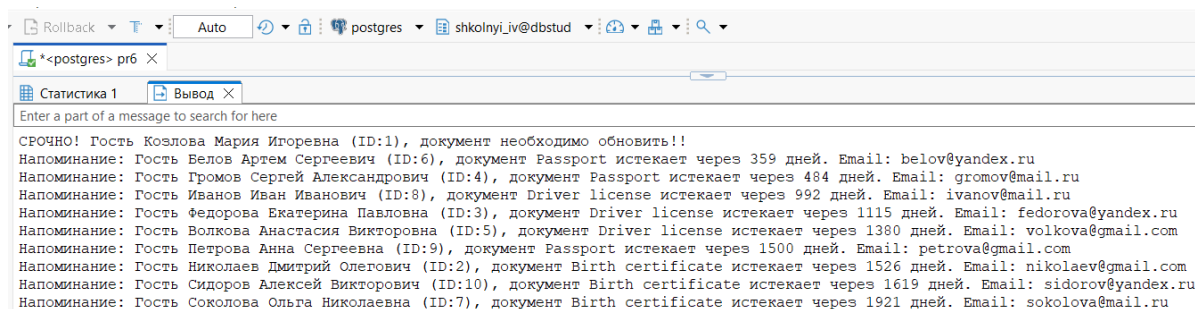


Рисунок 10 — Явный курсор

Займемся неявным курсором. Пусть есть задача формирования детального отчета по доходам каждого отеля с расчетом дополнительных метрик.

Листинг 13. Неявный курсор

```

-- Неявный курсор: Формирование отчета по доходам отелей
CREATE OR REPLACE FUNCTION get_hotel_revenue_report()
RETURNS TABLE(
    hotel_name VARCHAR(30),
    total_revenue DOUBLE PRECISION,
    average_booking_value DOUBLE PRECISION,
    total_bookings BIGINT,
    revenue_per_room DOUBLE PRECISION
) AS $$
DECLARE
    hotel_record RECORD;
    hotel_rooms_count INT;
BEGIN
    -- Цикл FOR автоматически управляет курсором
    FOR hotel_record IN
        SELECT
            h.id,
            h.name,
            h.star_rating
        FROM hotel h
        ORDER BY h.star_rating DESC, h.name
    LOOP
        -- Заполняем выходные параметры
        hotel_name := hotel_record.name;
    
```

```

-- Рассчитываем основные метрики
SELECT
    COALESCE(SUM(ci.total_bill), 0),
    COALESCE(AVG(ci.total_bill), 0),
    COUNT(ci.id)
INTO
    total_revenue,
    average_booking_value,
    total_bookings
FROM check_in ci
JOIN room r ON ci.room_id = r.id
WHERE r.hotel_id = hotel_record.id;

-- Дополнительная метрика: доход на номер
SELECT COUNT(*) INTO hotel_rooms_count
FROM room
WHERE hotel_id = hotel_record.id;

IF hotel_rooms_count > 0 THEN
    revenue_per_room := total_revenue / hotel_rooms_count;
ELSE
    revenue_per_room := 0;
END IF;

-- Возвращаем строку результата
RETURN NEXT;
END LOOP;

RETURN;
END;
$$ LANGUAGE plpgsql;

```

Протестируем курсор.

Листинг 14. Тестирование неявного курсора

```

SELECT
    hotel_name as "Отель",
    total_revenue as "Общий доход",
    average_booking_value as "Средний чек",
    total_bookings as "Кол-во броней",
    revenue_per_room as "Доход на номер"
FROM get_hotel_revenue_report()
ORDER BY total_revenue DESC;

```

Получаем следующий результат.

Rollback

T

Auto

↺

🔒

postgres

shkolnyi_iv@dbstud

📄

📄

🔍

<postgres> pr6

Результат 1

SELECT hotel_name as "Отель", total_revenue as "Обл" Введите SQL выражение чтобы отфильтровать результаты

Таблица

	AZ Отель	123 Общий доход	123 Средний чек	123 Кол-во броней	123 Доход на номер
1	Park Inn	231 500	77 166,6666666667	3	57 875
2	River Side	172 200	43 050	4	43 050
3	City Lights	114 300	28 575	4	28 575

Рисунок 11 — Корректная работа неявного курсора

ВЫВОД

В ходе выполнения практической работы были освоены навыки создания триггеров и курсоров.